ReasonLink: Characterizing and Optimizing the Handoff Between Neural and Symbolic Accelerators

Vayun Gupta (vg2684), Aaron Mano Cherian (amc2535)
COMS 6998: AI-Native Computing | Fall 2026
P1: Exploration & Progress | Due October 26, 2026, 11:59 PM ET

I. Project Direction and Updates

Neurosymbolic AI is an emerging compositional paradigm that fuses neural learning with symbolic reasoning to enhance the transparency, interpretability, and trustworthiness of AI. It also exhibits higher data efficiency making it promising for edge deployments [1]. Because the neural stage is compute bound, and the symbolic stage is memory bound, heteregenous hardware deployment, such as a specialized accelerator for each stage, is a natural fit. However, the interface between the two accelerators can become a bottleneck, limiting end-to-end performance.

We therefore ask: *after both ends of a neuro-symbolic pipeline are accelerated, does the interface between them become the bottleneck, and if so, which interface mechanisms remove it?*

In visual Sudoku, a neural network reads the given digits and a SAT solver completes the grid [2]. SAT-Accel's host interface [3] requires the CPU to rebuild and transfer roughly 632 KB of solver data per puzzle, including the unchanged Sudoku rules and their metadata. In our lean serial software baseline, the solver waits about 0.71 ms for this preparation before the MiniSat backend runs for another 0.73 ms.

ReasonLink aims to replace this repeated image construction with compact solver-native literals and reusable base constraints, so inference and solver-input preparation can overlap. Predictions should enter the solver's input path as they are produced. The challenge is to manage buffer ownership and reset assignments and learned clauses between puzzles without corrupting the shared rules. Figure 1 shows the baseline and current design.

Since the proposal, we replaced Chipyard/Rocket/Gemmini with ESP [4]/Ariane/NVDLA *nv_small* (int8) to use the available server toolchain and existing NVDLA integration. The server lacks v++/XRT, so we built a custom SAT-Accel integration and driver flow using Vitis HLS. To avoid building a combined tile, NVDLA and SAT-Accel remain separate and exchange data through DRAM. *ReasonLink Milestone 1* reuses the base image in DRAM; *Milestone 2* will keep the base on chip.

II. Exploration and Findings

Software optimization and overlap. We compared serial and double-buffered paths using *faithful* handling (clear and copy all 10.5 MB of allocated buffers) and *lean* handling (only the used regions, about 632 KB). Double buffering prepares puzzle $i{+}1$ while solving puzzle i. The pilot uses replayed classifier scores and MiniSat [5] on x86 under WSL2.

TABLE I
RTL cycles at 100 MHz; testbench memory has 30-cycle read latency and one burst at a time. Busy includes loading and search.

		ReasonLink		SAT-Accel busy	
Puzzle	Givens	Conv.	Commit	Baseline	RL
0 (SAT)	36	2,412	1,181	202,894	202,558
3 (SAT)	27	1,809	1,181	207,411	207,176
5 (UNSAT)	29	1,943	1,181	188,484	188,177

Lean handling reduces serial wall time from 2.81 to 1.45 ms (Fig. 2). Argmax takes less than 1 μs; image construction dominates conversion. Double buffering reduces lean wall time to 0.80 ms, leaving only 5.1% outside backend execution, but barely helps faithful mode. These results make lean double buffering a necessary comparison: overlap already hides most preparation in this pilot, while still rebuilding and copying the image.

Base reuse. The base formula has 729 variables and 11,988 clauses. Across 1,000 puzzles, only 1,212 image bytes (0.19%) differ from the base on average. SAT-Accel already applies unit clauses as level-0 fixed decisions. Supplying givens this way on a reusable DRAM image preserves SAT/UNSAT results on 1000/1000 puzzles. We therefore include CPU-only delta updates to separate the benefit of base reuse from hardware literal generation.

RTL loading cost. ReasonLink Milestone 1 passes three RTL puzzles with reference results and unchanged search counts. Conversion takes 67 cycles per given and commit takes 1,181 cycles (Table I). Across these cases, SAT-Accel busy cycles change by less than 0.2%: it still reads about 650 KB each solve. Milestone 1 removes CPU image construction but retains this loading cost. That motivates Milestone 2: keep the base on chip and roll back per-puzzle state.

Workload limits. Given-cell accuracy is 99.26%, but only 79.0% of puzzles recover the ground-truth grid; comparisons must hold predictions fixed. The current puzzles need only 1.17 mean MiniSat decisions, so harder instances are needed to test whether gains persist when search takes longer.

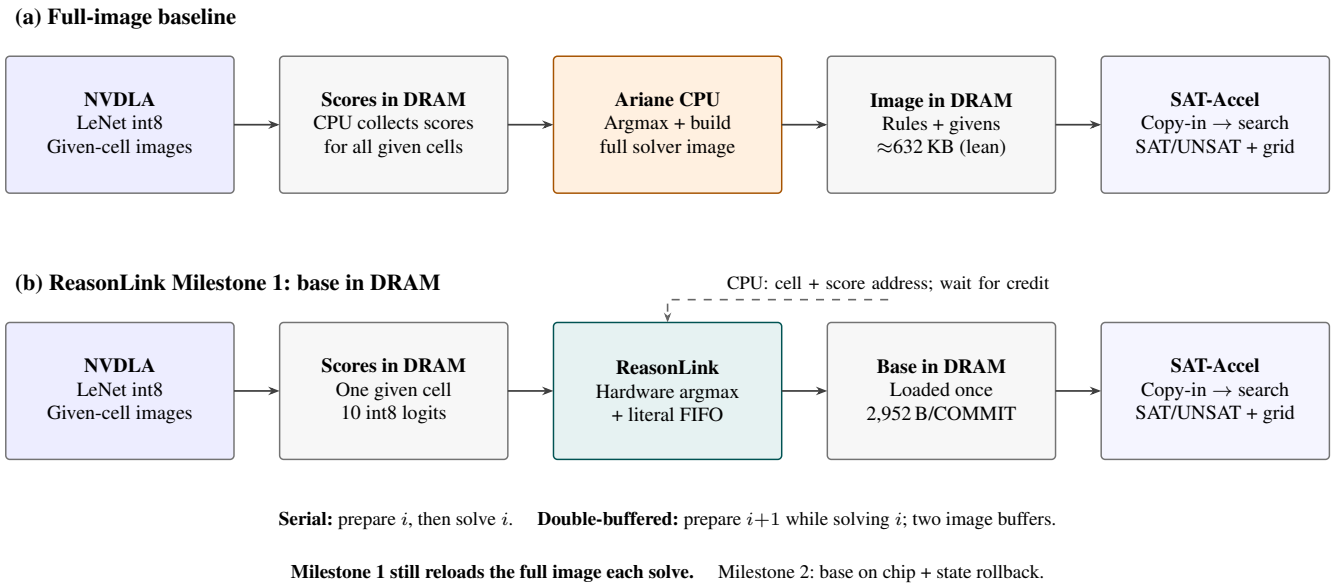


Fig. 1. ESP data paths. The baseline rebuilds the image every puzzle; ReasonLink Milestone 1 updates the fixed-decision array and counters (2,952 B including padding). Solid arrows carry data; dashed arrows indicate CPU control. The software pilot replays scores with MiniSat; RTL tests replay scores with SAT-Accel.

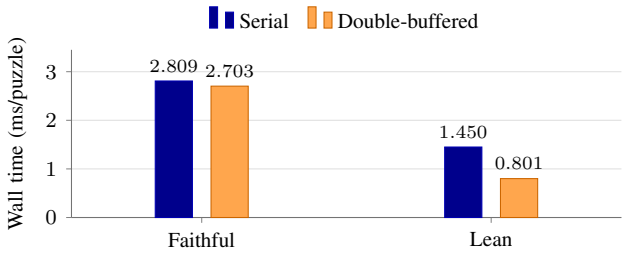


Fig. 2. Average wall time over 1,000 puzzles after 20 warm-ups; one run per configuration. Replayed scores and MiniSat on x86. Lean double buffering provides 1.81× throughput over lean serial execution.

III. Implementation and Infrastructure Progress

Software. We built 1,000 MNIST-based Sudoku puzzles, an int8 classifier emulator, a PySAT [6] reference, and the baseline harness. Retraining raised emulated MNIST accuracy from 74.5% to 99.20%; the NVDLA model compiles but awaits hardware validation. The C image builder matches SAT-Accel's host code byte for byte on 13 cases. All four pilot configurations agree on all 1,000 puzzles; eight solutions differ from the reference but satisfy their instances.

Hardware. Seven HLS kernels, a Vivado block design, APB/AXI shell, and C driver replace v++/XRT. We halved clause capacity to fit the VCU118. SAT-Accel passes 11 Sudoku and six other RTL cases. Its out-of-context netlist uses 112k LUTs and 434 URAMs (45%), with +2.0 ns post-synthesis slack at 10 ns. ReasonLink Milestone 1 passes RTL tests but has no synthesized netlist yet.

Integration and blockers. The 2×3 ESP SoC, Linux image, drivers, and board harness build. The harness passes host tests with a simulated device. Full-SoC simulation boots Ariane, discovers SAT-Accel, and loads an image; solve completion is pending. SoC synthesis passes, but an unlinked SAT-Accel checkpoint blocks bitstream implementation. No board timing or classifier-output validation is available. Code, results, and reproduction scripts are maintained in the ReasonLink repository.

IV. Next Steps

November 6 midterm. Present the pipeline, software comparisons, and Milestone 1 RTL results. Fix checkpoint linking, build the baseline bitstream, then synthesize ReasonLink and build its bitstream. Repeat pilot measurements to assess variability.

November 9 course P2. Run full-image, CPU-delta, and ReasonLink handoffs, each serial and double-buffered, with NVDLA and SAT-Accel on the VCU118. Check logits against the emulator and report inference, conversion, transfer, synchronization, solver busy cycles, and puzzles/s. This will measure whether hardware improves on optimized software.

Then implement ReasonLink Milestone 2, validate state rollback across puzzles, and test harder instances. The December 18 final submission will include code, pinned environments, raw results, and reproduction scripts.

REFERENCES

[1] Z. Wan, H. Yang, R. Raj, C.-K. Liu, A. Samajdar, A. Raychowdhury, and T. Krishna, "CogSys: Efficient and scalable neurosymbolic cognition system via algorithm-hardware co-design," in *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025. [Online]. Available: https://zishenwan.github.io/publication/HPCA25_CogSys.pdf

[2] P.-W. Wang, P. L. Donti, B. Wilder, and J. Z. Kolter, "SATNet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[3] M. Lo, M.-C. F. Chang, and J. Cong, "SAT-Accel: A modern SAT solver on a FPGA," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 234–246. [Online]. Available: https://github.com/milo168/FPGA25_SAT_Accel

[4] P. Mantovani, D. Giri, G. Di Guglielmo, L. Piccolboni, J. Zuckerman, E. G. Cota, M. Petracca, C. Pilato, and L. P. Carloni, "Agile SoC development with open ESP," in *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*, 2020. [Online]. Available: https://www.esp.cs.columbia.edu

[5] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Theory and Applications of Satisfiability Testing (SAT)*, ser. LNCS, vol. 2919, 2003, pp. 502–518.

[6] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python toolkit for prototyping with SAT oracles," in *Theory and Applications of Satisfiability Testing (SAT)*, 2018, pp. 428–437.